

03-语义分析与中间代码复习讲义

目录

1	03 语义分析与中间代码复习讲义
---	------------------

2

1 03 语义分析与中间代码复习讲义

对应资料:

- Slides/Slides/Ch6-1-Intermediate Code Generation.pdf
- Slides/Slides/Ch6-2-Translation into SSA.pdf
- Slides/Slides/Slides-Ant/7-symtable-handout.pdf
- Slides/Slides/Slides-Ant/8-type-systems-out.pdf
- Slides/Slides/Slides-Ant/9-semantics-ag-handout.pdf
- Slides/Slides/Slides-Ant/10-llvm-handout.pdf
- Slides/Slides/Slides-Ant/11-ir-expr-control-handout.pdf
- Slides/Slides/handout/3 IR SSA.pdf
- Review/compile.pdf
- Exam/2025.pdf 支配与 SSA 相关题
- Exam/过时/2022-exam-A.pdf 题目 5、6
- Exam/过时/2023-exam-A.pdf 题目 5
- Exam/过时/2021-exam-A.pdf 题目 4、5

1.1 一、考试目标

本章高频题:

语法制导定义: 设计属性和语义规则

中间代码: 写三地址码

回填: `truelist`、`falselist`、`nextlist`

SSA: 支配、支配边界、`phi`

零基础顺序:

先会写属性表, 再会写回填模板, 最后背 SSA 定义。

1.2 二、语义分析

语义分析检查语法树中的“含义”：

- 类型检查。
- 作用域检查。
- 声明与使用匹配。
- 函数调用参数个数和类型检查。

符号表常用操作：

```
enterScope()
exitScope()
define(name, type, attributes)
lookup(name)
```

1.3 三、语法制导定义 SDD

SDD 把属性和语义规则挂在产生式上。

两类属性：

综合属性：由子节点算父节点。

继承属性：由父节点或左侧兄弟节点传给当前节点。

答题格式：

属性定义：

X.val : 综合属性，表示数值

X.len : 综合属性，表示长度

产生式

$S \rightarrow L . L$

$L \rightarrow L B$

$B \rightarrow 0$

$B \rightarrow 1$

语义规则

$S.val = L1.val + L2.val / 2^{(L2.len)}$

$L.val = L1.val * 2 + B.val; L.len = L1.len + 1$

$B.val = 0$

$B.val = 1$

1.4 四、SDD 真题模板

二进制小数转十进制：

```

S -> L1 . L2    S.val = L1.val + L2.val / 2^(L2.len)
S -> L           S.val = L.val
L -> L1 B        L.val = L1.val * 2 + B.val; L.len = L1.len + 1
L -> B           L.val = B.val; L.len = 1
B -> 0           B.val = 0
B -> 1           B.val = 1

```

统计声明个数:

```

P -> D           P.count = D.count
D -> D1 ; D2      D.count = D1.count + D2.count
D -> T id         D.count = 1
D -> func id {D1} D.count = 1 + D1.count
T -> int          不改变 count

```

打印变量嵌套深度:

```

P -> D           D.depth = 0
D -> D1 ; D2      D1.depth = D.depth; D2.depth = D.depth
D -> T id         print(id, D.depth)
D -> func id {D1} D1.depth = D.depth + 1
T -> int          不改变 depth

```

C 声明英文说明模板:

```

D -> id          D.name = id.lexeme; D.desc = ""
D -> D1 ( )      D.name = D1.name; D.desc = D1.desc + " function returning"
D -> D1 [ ]      D.name = D1.name; D.desc = D1.desc + " array"
D -> * D1        D.name = D1.name; D.desc = D1.desc + " pointer to"
D -> ( D1 )      D.name = D1.name; D.desc = D1.desc
输出: declare D.name as D.desc

```

1.5 五、三地址码 TAC

三地址码每条指令最多包含三个地址。

常见形式:

```

x = y op z
x = op y
x = y
goto L
if x relop y goto L1
if x relop y goto L1 else goto L2
param x
call f, n
return x

```

数组访问在考试中可简化为表达式：

```
if dict[mid] < key goto L
```

1.6 六、回填

回填用于控制流翻译。先把跳转目标留空，之后统一填标签。

核心属性：

```
B.truelist  : B 为真时要跳转的指令编号集合
B.falselist : B 为假时要跳转的指令编号集合
S.nextlist  : S 执行完后要跳转的指令编号集合
```

核心函数：

```
makelist(i)    创建只含 i 的链表
merge(p, q)    合并链表
backpatch(p,L) 把 p 中所有空目标填成 L
nextquad       下一条指令编号
```

布尔表达式：

```
B -> E1 relop E2
B.truelist = makelist(nextquad)
emit("if E1 relop E2 goto _")
B.falselist = makelist(nextquad)
emit("goto _")
```

1.7 七、while 回填模板

语句：

```
while B do S1
```

模板：

```
M1.quad = nextquad      循环条件入口
翻译 B
M2.quad = nextquad      循环体入口
backpatch(B.truelist, M2.quad)
翻译 S1
emit("goto M1.quad")
backpatch(S1.nextlist, M1.quad)
S.nextlist = B.falselist
```

二分查找与 GCD 都按这个模板：

循环条件为真 -> 进入循环体
 循环体内部 **if/else** 继续回填
 循环体结束 -> **goto** 条件入口
 循环条件为假 -> 跳出 **while**

1.8 八、回填最小例子：GCD

源程序：

```
while a != b do
  if a > b then
    a = a - b
  else
    b = b - a
```

一种编号写法：

```
100: if a != b goto 102
101: goto 108
102: if a > b goto 104
103: goto 106
104: a = a - b
105: goto 100
106: b = b - a
107: goto 100
108: ...
```

说明：

B.truelist 回填到 102。
B.falselist 成为 **while** 的 **nextlist**，回填到 108。
if 的真分支到 104，假分支到 106。
 两个分支结束都回到 100。

1.9 九、SSA

SSA: Static Single Assignment, 每个变量只被定义一次。

普通代码：

```
x = 1
x = x + 1
```

SSA:

```
x1 = 1
x2 = x1 + 1
```

控制流汇合处使用 phi:

```
if ...
    x1 = 1
else
    x2 = 2
x3 = phi(x1, x2)
```

phi 含义:

根据控制流来自哪个前驱块, 选择对应版本。

1.10 十、支配关系

定义:

Dom(n): 从入口到 **n** 的每条路径都经过的节点集合。
idom(n): **n** 的最近严格支配者。
Dom Frontier(n): **n** 支配某个前驱, 但不严格支配该节点的位置。
PostDom(n): 从 **n** 到出口的每条路径都经过的节点集合。

SSA 中 phi 放置:

变量 **x** 在多个基本块中被定义。
在这些定义块的支配边界处放置 **x** 的 **phi**。

支配树:

除入口外, 每个节点有唯一 **idom**。
idom 关系无环。
因此支配关系图是以入口为根的树; 多入口时是森林。

1.11 十一、自测题

题 1: 综合属性和继承属性区别?

答案:

综合属性由子节点向父节点传递。

继承属性由父节点或左侧兄弟节点向当前节点传递。

题 2: 回填三件套是什么?

答案:

`makelist`、`merge`、`backpatch`。

题 3: `while` 的假出口是什么?

答案:

条件 B 的 `falselist`。

题 4: SSA 中 `phi` 放在哪里?

答案:

放在变量定义块的支配边界处。